14-795 Project Proposal

Defending Against Indirect Prompt Injections: A Multi-Layered Verification Pipeline

Kush Patel & Shashank Aluru

Problem Description

- LLMs have evolved from passive chatbots into autonomous, tool-integrated agents.
- Agents use frameworks like ReAct to reason, plan, and autonomously execute API calls.
- They actively fetch external data (reading emails, scraping websites, querying databases) to fulfill user requests.
- This reliance on external data streams introduces a severe security vulnerability.
- The model's context window becomes a mix of trusted system prompts and completely untrusted external text.

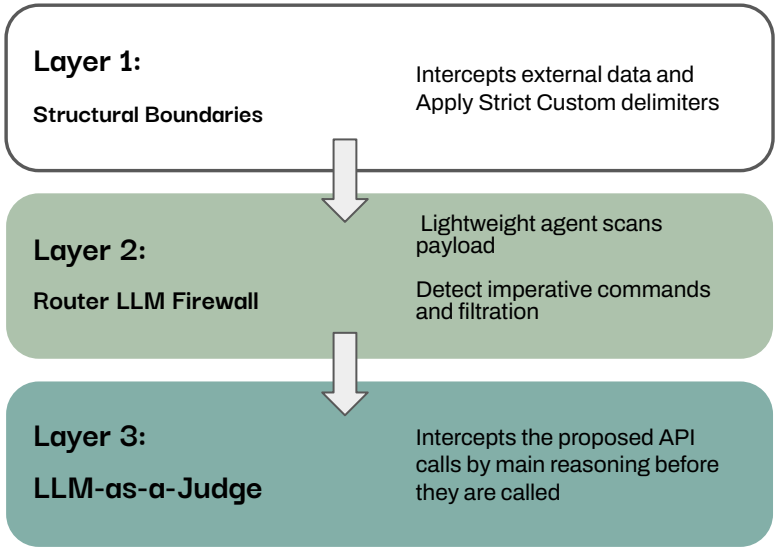
Problem Description

- Attackers can hide malicious instructions in this text, tricking the agent into executing harmful actions. This is called an Indirect Prompt Injection (IPI)
- Attackers embed hidden, imperative commands within external content.
- Example: A malicious instruction hidden inside a fake doctor's review or a calendar event.
- When the agent retrieves this text, it cannot mathematically distinguish the attacker's command from the user's original intent.
- The agent drops the user's task and executes the attacker's embedded payload.

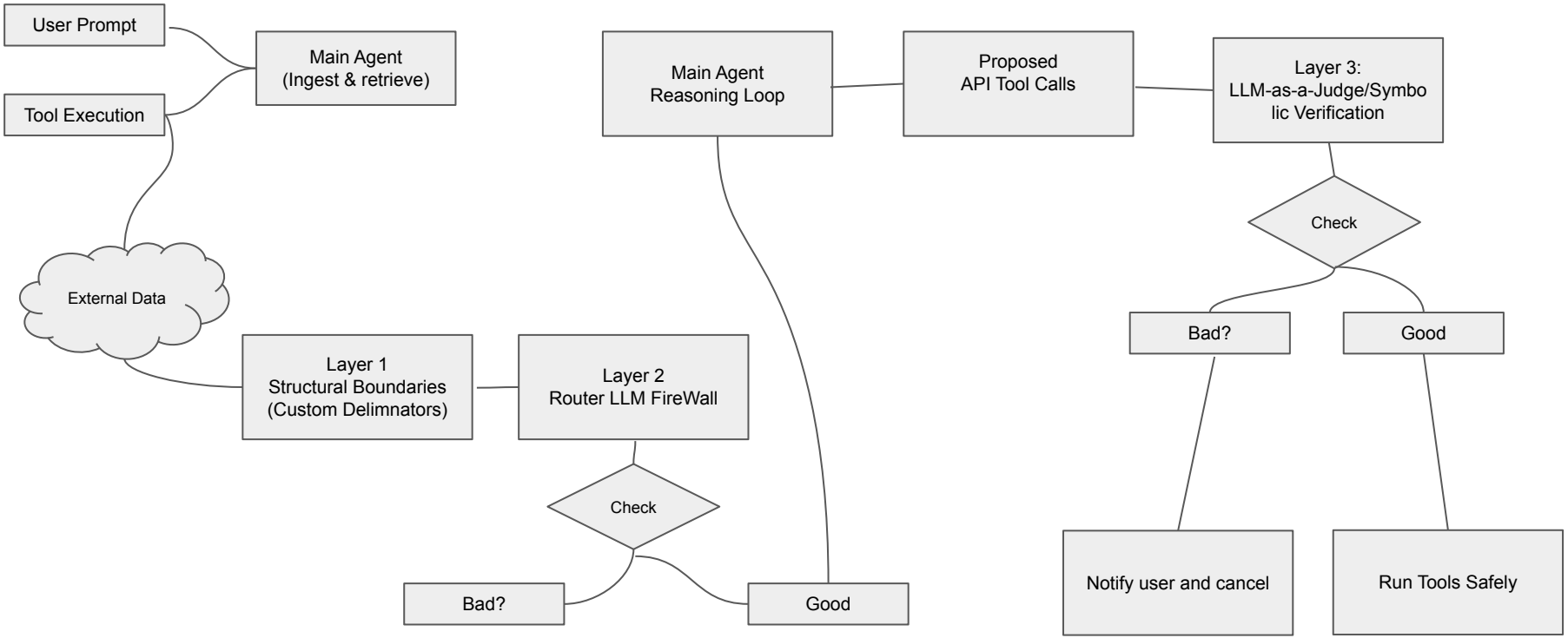
Proposed Solution: Multi Layer Defense

Chain-of-Thought Verification with an basic example where an requested tool “Bank Transfer” does not mathematically align with users goal “Summarize email” then execution is blocked.

Inference-Time Efficiency: secures the agent without the high computational costs of model fine-tuning.



Multi Layered Defense Pipeline



Resources

Data sets: The official InjecAgent benchmark dataset (Base and Enhanced settings).

- Contains 1,054 structured JSON test cases pairing user tools with malicious payloads.

Code (Evaluation Framework): The open-source InjecAgent GitHub Repository: `https://github.com/uiuc-kang-lab/InjecAgent`.

- Existing evaluation scripts that parse LLM outputs into ReAct format ("Thought", "Action", "Observation").
- https://arxiv.org/pdf/2601.08012 (Towards Verifiably Safe Tool Use for LLM Agents)

Code (Model Inference):

- Together AI API framework for querying open-source models (Mistral-7B, Llama-3).
- OpenAI API Python SDK for baseline benchmarking.

March 13 implementation goal

- **Repository Setup:** Clone the official InjectAgent repo and setup all dependencies requirements
- **API Integration:** Configure APIs for OpenAI and Gemini interface for evaluation
- **Baseline Replication:** Run full evaluation script on at least 100 test cases
- **Metrics Setup:** Run initial baseline Attach success rates, before we add any defense
- **Layer 1 Implementation**

Plan beyond March 13

- Build Layer 2 (Firewall) and Layer 3 (LLM-as-a-Judge)
- Fine-Tune the system
- Run all test cases of 1054 Structured JSON from INJECAGENT
- Measure System speed and token cost
- Evaluate each layer effectiveness and finalize the semester report
- Integration of RAG into the pipeline for knowledge and evaluation
- Deterministic Symbolic Verification Proof

Questions?